



5

18

2700 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية



B-Tree

د. مدحت الصّوص

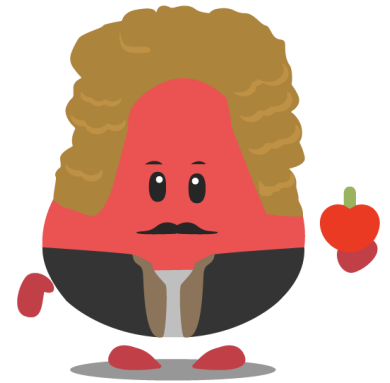
RB Informatics; 10/05/2026

خوارزميات ٢

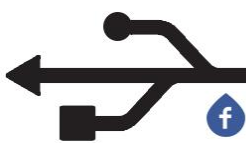
تحدثنا في المحاضرة السابقة عن الـ AVL و اليوم سنتحدث عن نوع آخر من الأشجار، لنكتشفه معاً 😊

*“Once you stop learning...
You start dying”*

- RBCs' newton



أهم العناوين الواردة في المحاضرة	
رقم الصفحة	عنوان الفقرة
2	مفهوم الـ B-Tree
3	البحث في B-Tree
6	الإضافة في B-Tree
10	الحذف في B-Tree



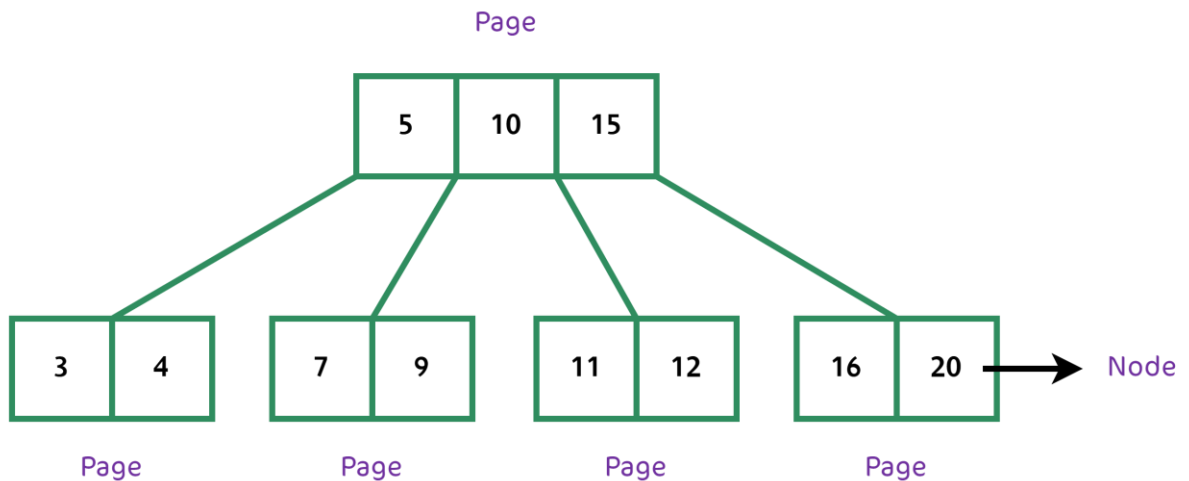
لنفرض أن لدينا شجرة كبيرة جداً مخزنة في الـ hard disk و فيها مليون عقدة، في أشجار الـ AVL كان تعقيد البحث عن عنصر معين هو $O(\log_2 n)$ فيكون التعقيد مساوياً لـ $\log_2 10^6 = 20$ تقريباً، أيّ سنحتاج إلى 20 مرة وصول إلى الـ hard disk.

و لنفرض أن لدينا شجرة أخرى فيها مليار عقدة فسيكون تعقيد البحث $\log_2 10^9 = 30$ تقريباً، لكن عند البحث في الشجرة كنا ننقل الشجرة إلى الـ RAM فهل من المعقول نقل مليار عقدة إلى الـ RAM؟؟
لذلك سنلجأ إلى إبقاء الشجرة في الـ hard disk و عند كل عملية مقارنة نذهب إلى الـ hard disk لإحضار العقدة التي نريد المقارنة معها، فظهرت مشكلة أكبر و هي أن عملية نقل العنصر من الـ hard disk إلى الـ RAM مكلفة جداً و بطيئة.

و لحل هذه المشكلة ظهر مفهوم الـ **B-Tree** و الذي تهدف إلى تقليل عدد مرّات الوصول إلى الـ hard disk، فبدلاً من الذهاب إلى الـ hard disk 20 أو 30 مرة سنقل عدد مرات الوصول إلى عدد أقل من ذلك بكثير.

B-Tree (bayer tree)

هي نوع من الأشجار المتوازنة (balanced tree) و هي أشجار معمّمة؛ أي ليست ثنائية.
و في هذا النوع لدينا عقدة و صفحة، حيث سنقوم بالنقل من الـ hard disk صفحة تلو الصفحة و ليس عقدة تلو العقدة.



و نلاحظ أن في الأشجار من نوع B_Tree الخصائص الآتية:

- لكل شجرة عدد ثابت t و يسمّى درجة الشجرة.
 - كل صفحة تحوي عدداً معيناً من العقد تحدده قيمة t بحسب الشروط الآتية:
1. عدد العقد في الصفحة الواحدة يكون محصوراً بين t و $2t$ ، باستثناء الجذر فعدد العقد فيه محصور بين t و $2t$ (لا يوجد حد أدنى عند الجذر أي على الأقل يوجد عقدة واحدة).

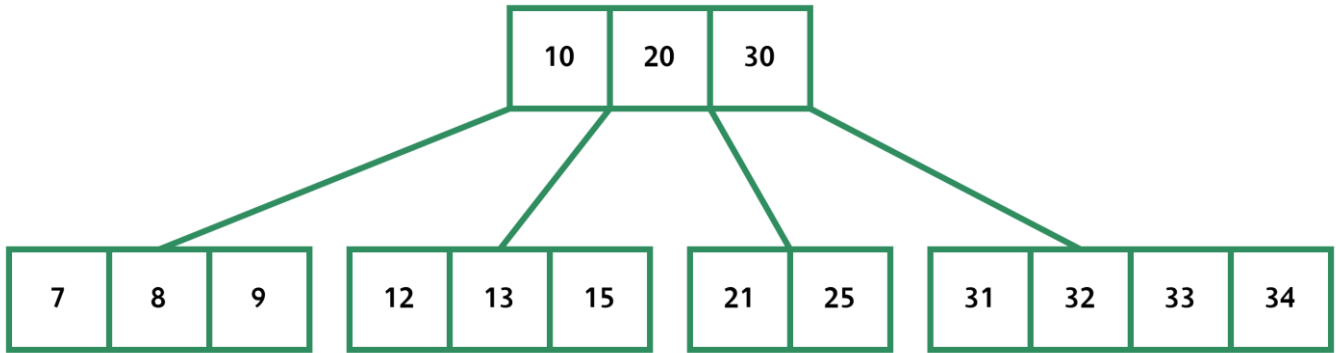
2. تكون العناصر (العقد) ضمن الصفحة مرتبة (sorted) تنازلياً أو تصاعدياً.

3. كل صفحة تحتوي على m عقدة سيكون لها إما $m+1$ ابن أو تكون بلا أبناء أي هي ورقة.

4. العقد ضمن الصفحة تحقق شرط الترتيب مع الصفحة الأب.

5. جميع أوراق الشجرة تقع ضمن نفس المستوى و هو المستوى الأخير.

مثال: لدينا B-Tree درجة الشجرة هي $t = 2$:



و نلاحظ أن الشجرة تحقق الشروط السابقة:

- عدد العناصر في كل صفحة محصور بين $[t, 2t]$ أي بين $[2, 4]$.
- كل الأوراق موجودة في نفس المستوى.
- في الجذر (الصفحة الأب) $m = 3$ ، و عدد الأبناء هو: $m + 1 = 3 + 1 = 4$
- قيم عناصر الأبناء محصورة بين قيم عناصر آبائها فهي تحقق شرط الترتيب، حيث $\{7, 8, 9\}$ أصغر من الأب 10، وكذلك $\{12, 13, 15\}$ هي أكبر من 10 و أصغر من 20، و هكذا...

عملية البحث (Search in B-Tree)

البحث في الـ B-Tree يكون على مرحلتين:

1. البحث في صفحة واحدة:

بما أننا نبحث عن عنصر معين لذلك علينا المرور بالصفحة حتى نجد العنصر، ويمكن أن نبحث في الصفحة إما باستخدام البحث الثنائي أو باستخدام البحث الخطي.
وإن لم نجد العنصر ننتقل إلى الصفحة التالية.

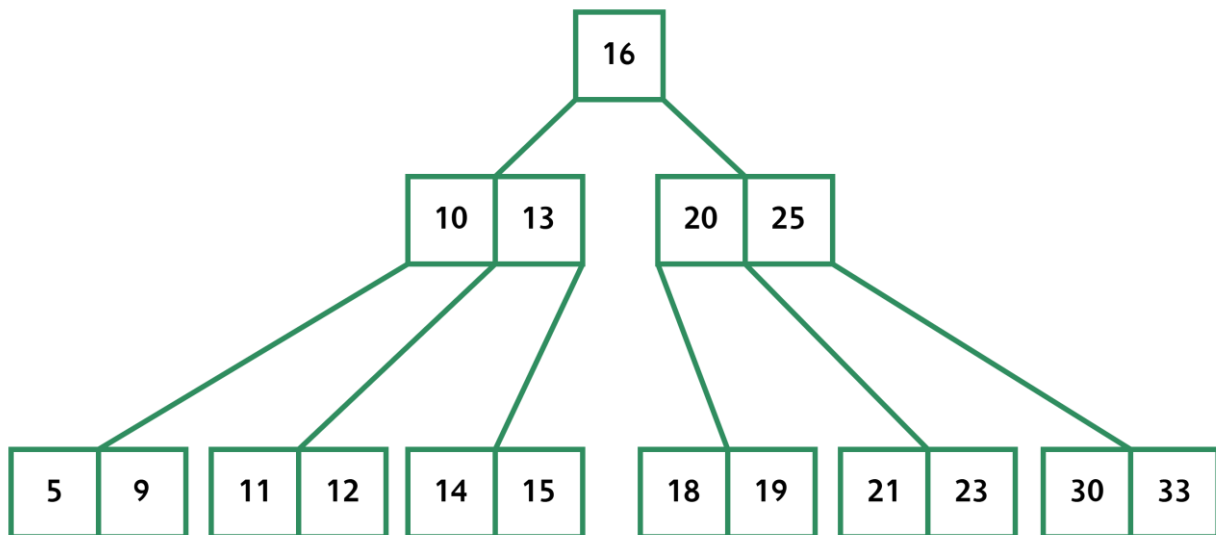
2. الانتقال للصفحة التالية:

إذا لم نجد العنصر فسيكون العنصر كآتي:

- إما أكبر من كل العناصر في الصفحة التي نبحث فيها.
 - أو أصغر من العناصر.
 - أو موجود بين عنصرين في الصفحة.
- وحسب الحالات السابقة ننتقل إلى الصفحة التالية.

ملاحظة:

البحث يبدأ من الـ Root، نقارن معه وحسب القيمة ننتقل إلى الصفحة التالية ونبحث داخلها عن هذا العنصر، فإما نجده في الصفحة الحالية أو ننتقل إلى الصفحة التالية.



البحث عن العدد (21):

نقارن مع الجذر ونجد أن $16 < 21$ لذلك نذهب إلى يمين الجذر.

الصفحة على يمين الجذر هي [20, 25]، نبحث في هذه الصفحة عن العنصر (21) ونجد أن العنصر غير موجود ولكن $25 > 21 > 20$ لذلك نذهب إلى الابن الذي بين (25) والـ (20).

الصفحة التي بين الـ (20) والـ (25) هي [21, 23] وعند البحث في هذه الصفحة نجد العنصر (21).

حساب تعقيد البحث

في الواقع التعقيد يعتمد على:

1. العمليات ضمن الصفحة:

الصفحة فيها من t إلى $2t$ عقدة، في أسوأ الأحوال $2t$ ؛ إذاً البحث:

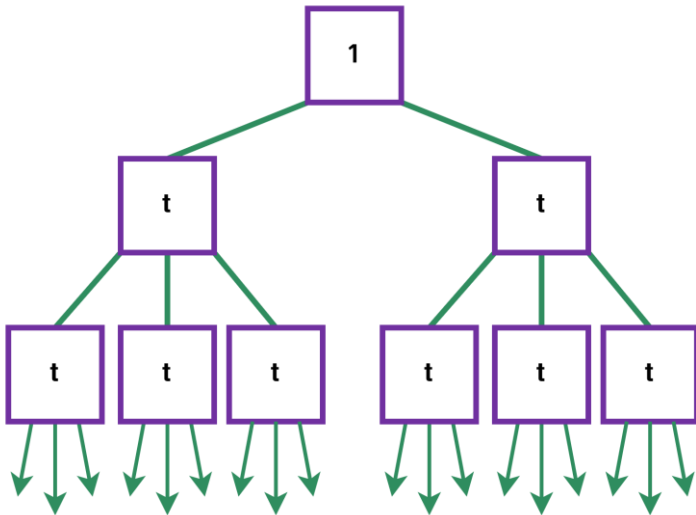
- إما بحث خطي (linear search) بكلفة $2t$ ؛ إذاً $O(t)$.
- أو بحث ثنائي (binary search) بكلفة $\log(2t)$ ؛ إذاً $O(\log(2t))$.

2. عدد الصفحات:

- عدد الصفحات التي سنبحث فيها: في أسوأ الأحوال يكون العنصر غير موجود أو في آخر صفحة؛ إذاً $O(h)$.
- التعقيد الكلي هو: $O(h \cdot \log(t))$ أو $O(h \cdot t)$.

لكننا نريد التعقيد بدلالة n وليس h ، لذا سنقوم بإيجاد علاقة بين n و h .

سنهتمّ بأسوأ حالة (الارتفاع h أكبر ما يمكن) حيث لدينا n عنصر ونريد توزيعه على B-Tree بأطول ارتفاع بالتالي نريد أكبر عدد من الصفحات و نضع في كلّ صفحة أقل عدد من العقد.



الارتفاع	عدد الصفحات	عدد العناصر بالمستوى
0	1	1
1	2	$2t$
2	$2(t+1)$	$2t(t+1)$
3	$2(t+1)^2$	$2t(t+1)^2$
...	...	...
h	$2(t+1)^{h-1}$	$2t(t+1)^{h-1}$

وهكذا...

لنحسب عدد العقد n و هي مجموع العناصر في كافة المستويات، أي: sum للعمود الأخير في جدولنا.

في أسوأ حالة يكون ارتفاع الشجرة أكبر ما يمكن وكل مستوى يحتوي على الحد الأدنى المسموح به من العناصر فيكون:

$$n = 1 + 2t + 2t(t+1) + 2t(t+1)^2 + \dots = 1 + 2t \sum_{i=1}^h (t+1)^{i-1}$$

و في الحالة العامة سيكون:

$$n \geq 1 + 2t \sum_{i=1}^h (t+1)^{i-1}$$

إذاً بأسوأ الحالات n تساوي هذا المقدار:

$$1 + 2t \sum_{i=1}^h (t+1)^{i-1}$$

بحيث أن $(t+1)^{i-1}$ هي متتالية هندسية أساسها $(t+1)$ و أيضاً $t \neq 0$ فيكون: $n \geq 1 + 2t \left(\frac{(t+1)^h - 1}{(t+1) - 1} \right)$

نقوم بإصلاح العبارة لنحصل على:

$$n \geq 1 + 2((t+1)^h - 1)$$

$$\frac{n+1}{2} \geq (t+1)^h$$

بأخذ لوغاريتم للطرفين نحصل على:

$$\log_{t+1} \left(\frac{n+1}{2} \right) \geq h$$

أي أن $h \approx \log_{t+1}(n) \approx \log_t(n)$

فيكون تعقيد عملية البحث مساوياً لـ: $O(h \cdot \log(t))$ إذا كان البحث ثنائياً أي: $O(\log(t) \cdot \log_t(n))$

أو $O(h \cdot t)$ إذا كان البحث خطياً أي: $O(t \cdot \log_t(n))$

و بما أن t ثابت نهملها، حيث أن تكلفة عمليات المقارنة ضمن الصفحة هي t ، وهي عمليات تحدث في الـ RAM بسرعة كبيرة مقارنةً بعملية النقل من الـ Hard Disk. و تكون صيغة التعقيد النهائية هي: $O(\log_t(n))$.

انتبه: قد تختلف القيم في المراجع بحسب تعريف t و حسب عدد المستويات لكن ما يهمنا هو معرفة كيف استنتجنا التعقيد النهائي.

عملية الإضافة Insert in B-Tree

الإضافة هنا تكون حصراً في الأوراق، و نميز حالتين:

1. الإضافة إلى صفحة مع عدم اختلال أحد الشروط:

نعلم بأن أول شرط لـ B-Tree هو أن يكون عدد العناصر داخل الصفحة محصور بين $[t, 2t]$ ، فإذا أضفنا عنصراً ومازالت الصفحة المضاف إليها تحقق الشرط لا نعدّل شيئاً على الشجرة (طبعاً الجذر فيه استثناء وهو أن عدد عناصره محصورة بين $[1, 2t]$).

2. الإضافة إلى صفحة مع اختلال أحد الشروط:

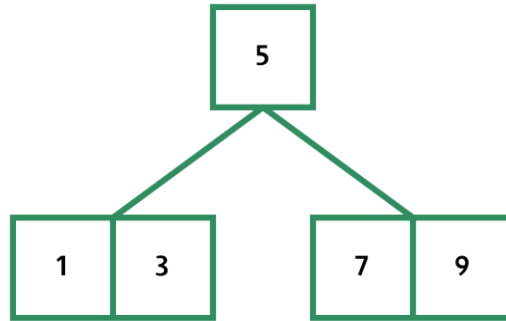
إن كان لدينا شجرة درجتها $t = 2$ والصفحة في المستوى الأخير تحتوي على (4) عناصر، فعند الإضافة لهذه الصفحة سيصبح عدد العناصر (5) وهو أكبر من $t = 4$ لذلك سيختل الشرط الأول، ولحل هذه المشكلة نطبق الخطوات التالية.

خطوات إضافة عنصر

بما أن الحد الأقصى لعدد العناصر في كل صفحة هو عدد زوجي (لأننا نضرب بـ 2، أي $2t$) فإنه عند إضافة عنصر زائد سيكون حجم المصفوفة دائماً فردي.

مثال لتوضيح الفكرة:

لنفرض أن الصفحة $[1, 3, 5, 7]$ موجودة بشجرة درجتها $t = 2$ ، أي أن هذه الصفحة تحتوي على أقصى عدد من العناصر فيها، وهو عدد زوجي بالتأكيد لأننا نضرب الـ t بـ (2).
إذا أضفنا العدد (9) لهذه الصفحة ستصبح $[1, 3, 5, 7, 9]$ ، أصبح حجم المصفوفة (5)، وهو أكبر من الحد المسموح به، ونلاحظ أن حجم المصفوفة أصبح فردياً.
← لذلك نأخذ العنصر الأوسط ونرفعه إلى المستوى الأعلى منه.



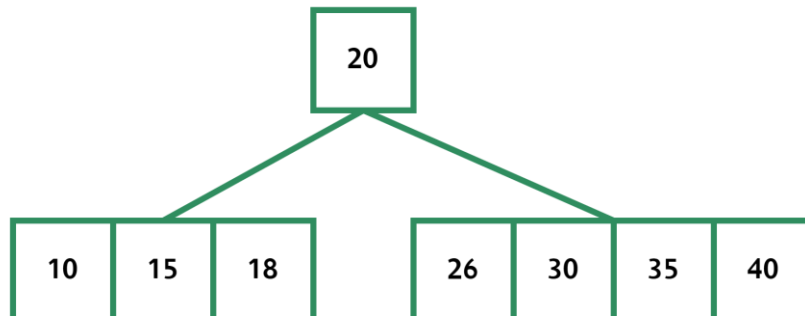
نلاحظ أننا جعلنا العنصر الأوسط هو الأب وقسمنا الصفحة إلى صفحتين، الصفحة اليسارية تحتوي على القيم الأصغر من (5)، والصفحة اليمينية تحتوي على القيم الأكبر من (5)، وبهذه الطريقة نضيف جميع العناصر إلى الشجرة.

ولكن ماذا لو كان الأب يحتوي على $2t$ عنصر ولا يمكن رفع العدد الأوسط إليه؟

في هذه الحالة سوف نكرر عملية الرفع للأب، وإذا واجه الأب نفس المشكلة نكرر عملية الرفع حتى نصل إلى الجذر، فإذا كان الجذر أيضاً يحتوي على $2t$ عنصر، عندها نفتح مستوى جديد فوق الجذر ونجعل العنصر الأوسط هو الجذر الجديد.

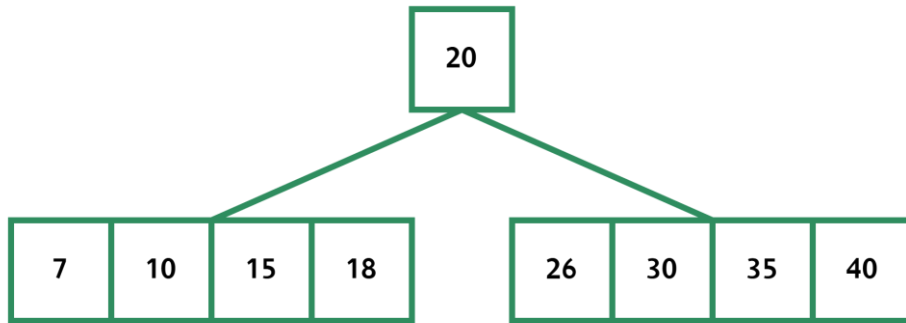
(ولهذا السبب هناك استثناء للجذر ليكون عدد قيمه محصور بين $[1, 2t]$).

مثال: ليكن لدينا الشجرة التالية من الرتبة $t = 2$:



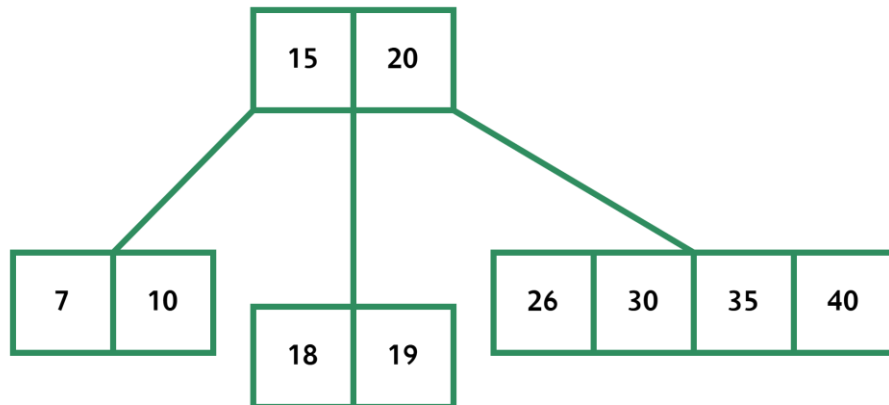
إضافة الرقم (7):

أولاً سنبحث عن المكان الملائم للرقم (7).
 نلاحظ أن (7) أصغر من (20) لذلك نذهب إلى اليسار.
 العنصر (7) هو أصغر عنصر في الصفحة لذلك يوضع كأول عنصر، فتصبح الشجرة بالشكل:

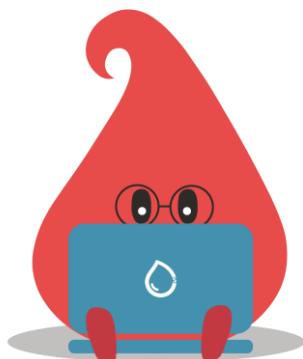


إضافة الرقم (19):

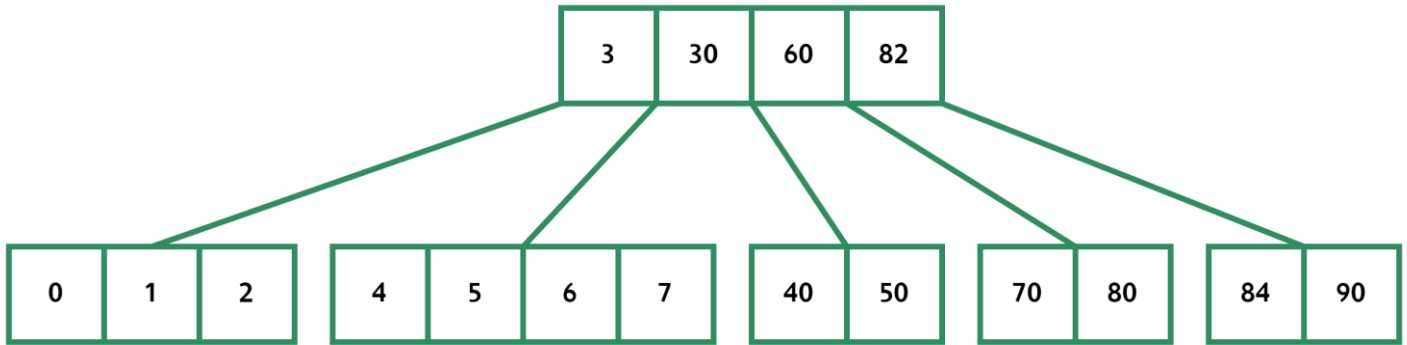
أولاً نبحث عن المكان الملائم، وهو على يسار الجذر.
 بعد الإضافة أصبحت الصفحة اليسارية تحتوي على عناصر أكثر من 2t هنا اختل شرط الشجرة؛ لذلك نأخذ العنصر الأوسط (15) ونرفعه إلى الأب ومن ثم نقسم الصفحة إلى قسمين، العناصر الأكبر من العنصر الأوسط تكون موصولة إلى يمينه والعناصر الأصغر منه تكون موصولة إلى يساره.



ملاحظة: في المثال السابق، بما أن الجذر أصبح فيه عنصرين، لذلك صار لديه (3) أبناء وذلك طبقاً للشرط الثالث في ال B-Tree.



مثال: لتكن لدينا الشجرة التالية من الرتبة $t = 2$:

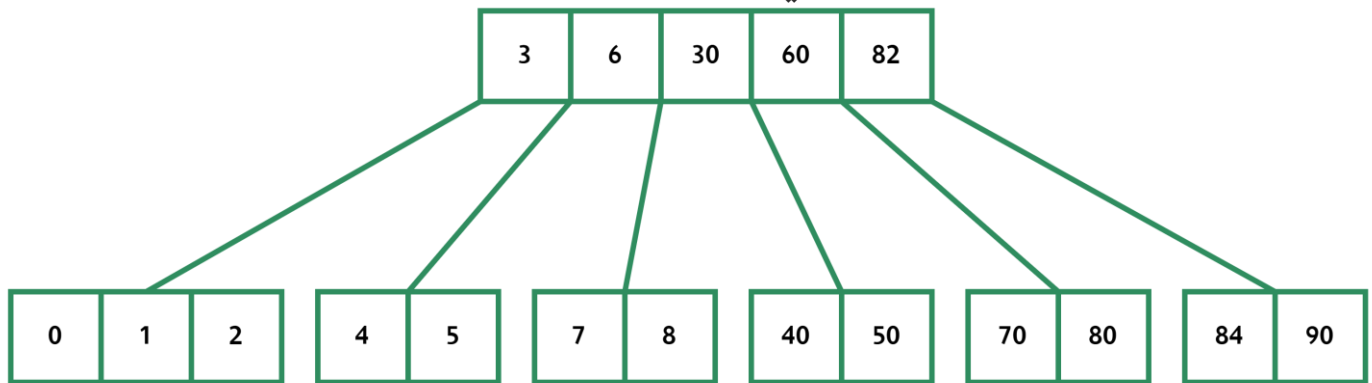


إضافة الرقم (8):

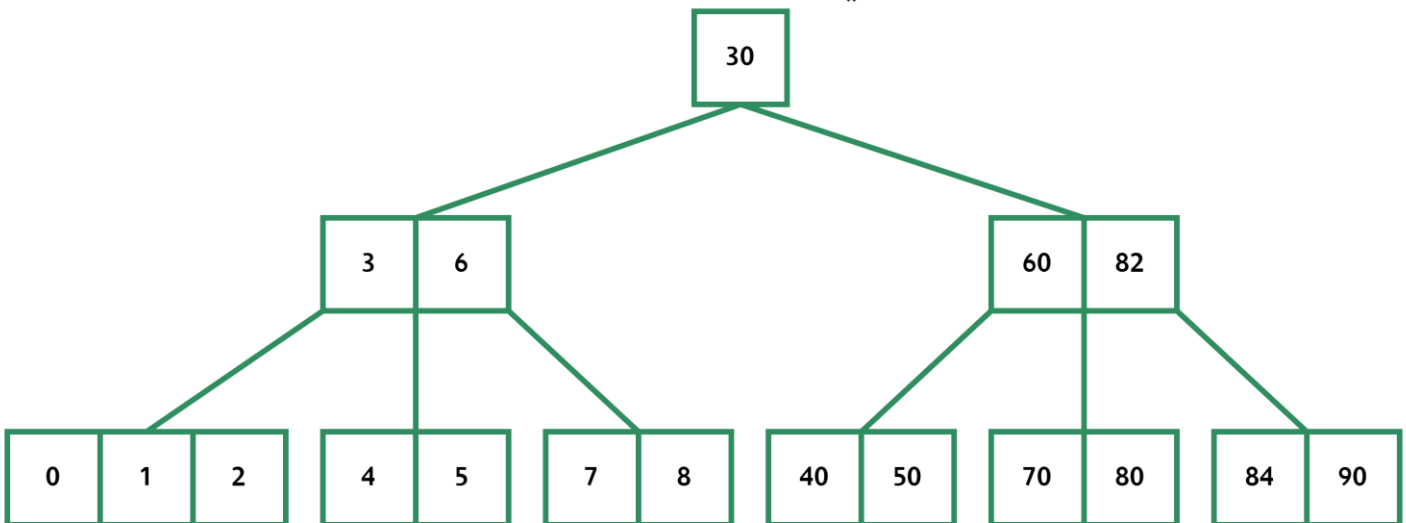
نبحث أولاً عن المكان المناسب للعنصر، فنجد أن $3 < 8 < 30$ لذلك تكون الصفحة ما بين (3) و (30).

أصبح في الصفحة عدد أكبر من $2t$ من 4 5 6 7 8.

← لذلك نرفع الأوسط للجذر وتصبح الشجرة كالآتي:



ولكن أصبح حجم الأب أكبر من $2t$ لذلك نكرر عملية الرفع مرّة أخرى، وبما أن الأب هو الجذر فسوف نفتتح مستوى جديد نضع فيه العنصر الأوسط، فتصبح الشجرة كالآتي:



و نلاحظ أن الشجرة الناتجة تحقق كل شروط ال B-Tree.

حساب تعقيد الإضافة:

بما أننا نبحث عن المكان الملائم لكل عقدة مضافة، فإن التعقيد يكون من نفس تعقيد البحث وهو:

$$O(\log(t) \cdot \log_t(n)) \text{ أو } O(t \cdot \log_t(n))$$

ومضافاً إليه $O(t)$ لأننا عند الإضافة نقوم بالإضافة في مصفوفة، إذ يمكننا تمثيل B-Tree على الحاسب باستخدام صفين (two classes).

الصف الأول للعقدة: فيه قيمة العقدة و سهم (يشير) للابن، والصف الثاني للصفحة: يحتوي على مصفوفة العقد يكون حجمها $2t$ دائماً و سهم (هو الوصلة من العقدة الأب).

كلفة الإصلاحات (إزاحة، رفع عقدة،...): جميعها من رتبة t أي لن تؤثر على التعقيد النهائي لأن t ثابت كما ذكرنا، إذاً التعقيد النهائي من رتبة: $O(\log_t(n))$.

عملية الحذف Deletion

نميز حالتين للحذف:

1. حذف ورقة، أي أن الحذف يتم على المستوى الأخير.

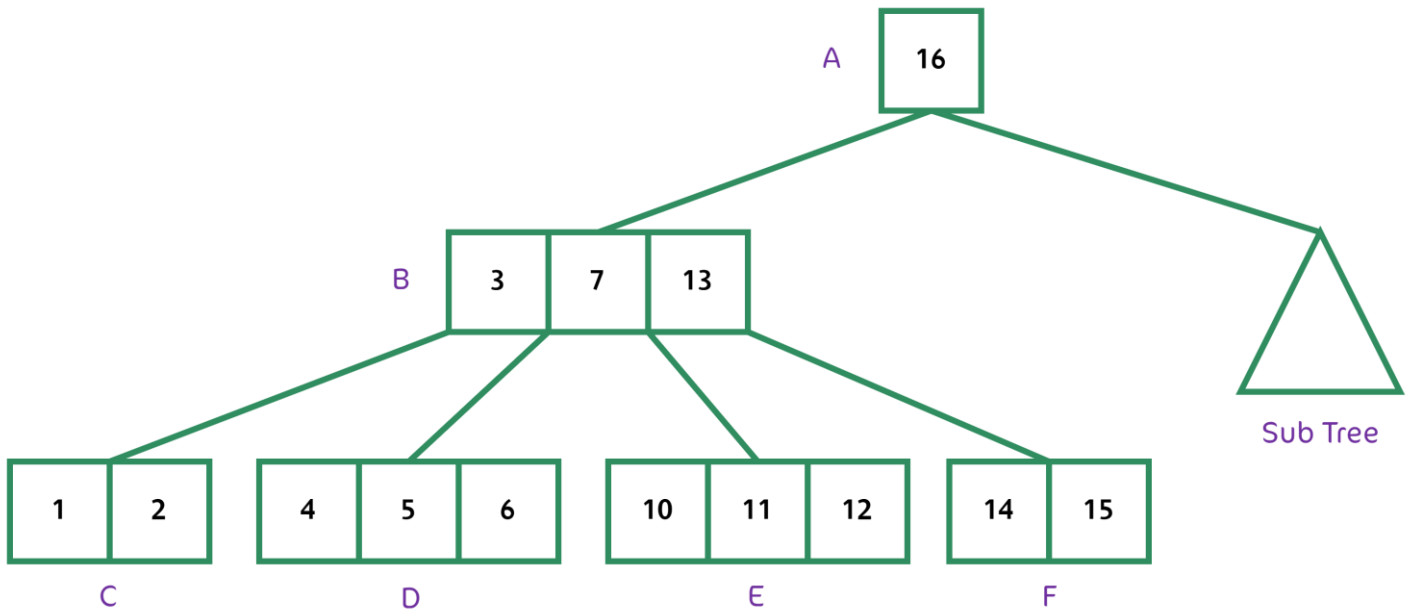
2. حذف من صفحة داخلية.

أولاً: الحذف من الورقة:

- الصفحة فيها أكثر من t عنصر: نحذف بكل بساطة.
- الصفحة فيها t عنصر: بهذه الحالة سيصبح بالصفحة أقل من t عنصر بالتالي اختل شرط B-Tree، فما الحل؟؟! نقوم باستعارة عنصر من الأب فيصبح عدد العناصر في الصفحة الابن t عنصر، ولكن سنواجه مشكلة أخرى: نعلم أن كل صفحة تحوي m عنصر ولها $m+1$ ابن وعند استعارة عنصر من الأب ستقل m بمقدار واحد وعدد الأبناء سيبقى $m+1$ ولحل هذه المشكلة سنميز حالتين:

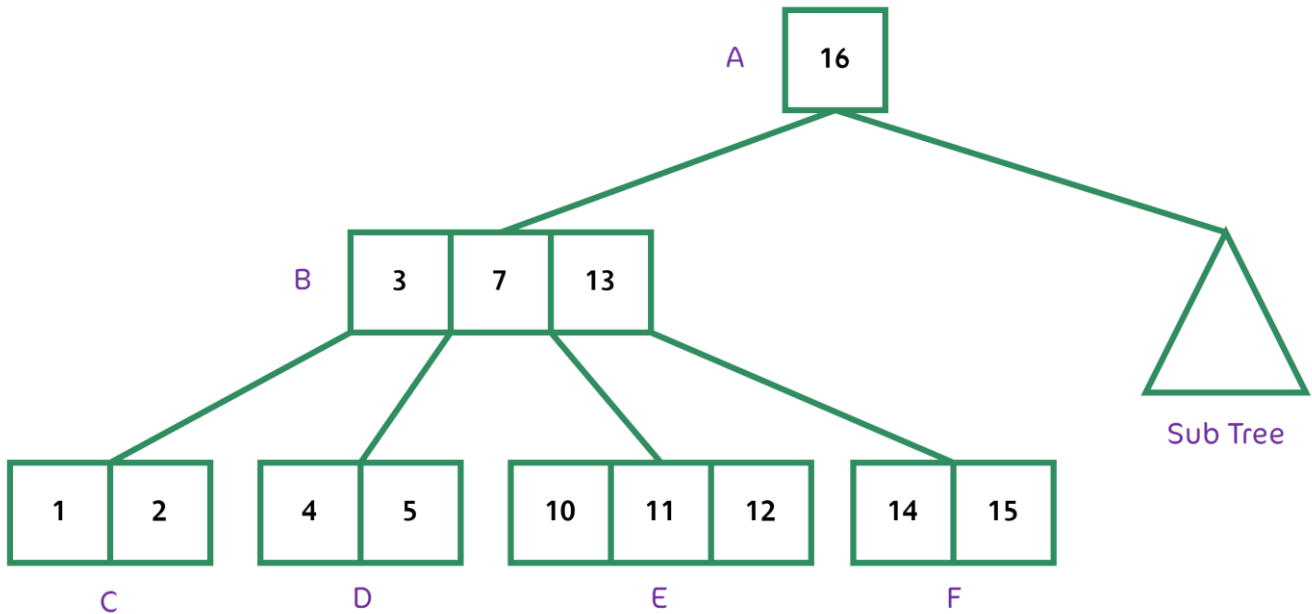
الحالة الأولى: إذا كان للصفحة الابن أخ يحوي أكثر من t عنصر سيقوم الأخ بتعويض الأب عن العنصر الذي فقده.

الحالة الثانية: إذا كان للصفحة الابن أخ يحوي t عنصر ندمج صفحة الابن مع الأخ، فصفحة الابن فيها t عنصر و صفحة الأخ فيها t عنصر إذاً الصفحة الجديدة تحوي $2t$ عنصر ولن يختل شرط الشجرة.

مثال توضيحي:لدينا B-Tree التالية حيث $t = 2$:

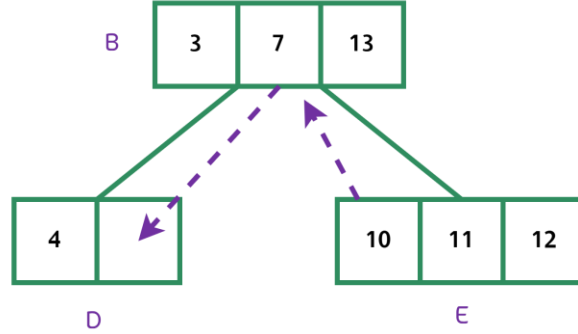
نريد حذف العنصر (6):

العنصر (6) موجود في الصفحة D و هو ورقة تحوي أكثر من t عنصر ← نحذف العنصر مباشرة و دون إجراء أي تغيير. و يصبح شكل الشجرة:

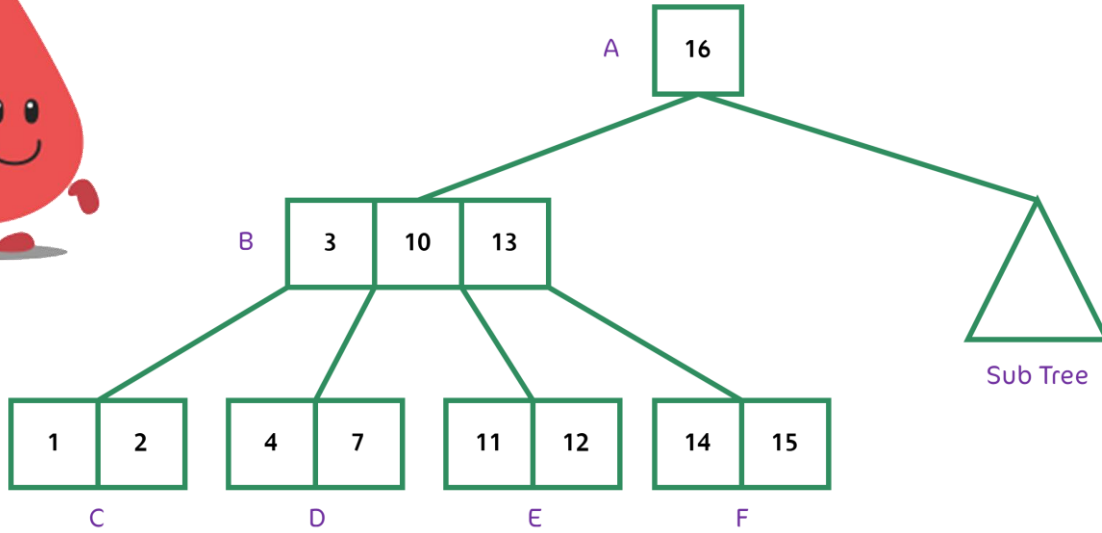
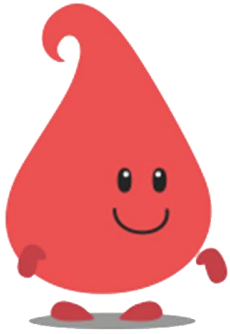


نريد حذف العنصر (5):

العنصر (5) موجود في الصفحة D و لكن هو ورقة تحوي t عنصر فقط ← بالتالي أخ الصفحة D هو E يحوي أكثر من t عنصر، فيمكننا التعويض عن طريق الأخ ← نضع مكان الـ (5) عنصر من الأب و سيقوم الأخ بتعويض الأب عن العنصر المفقود، كما في الشكل:

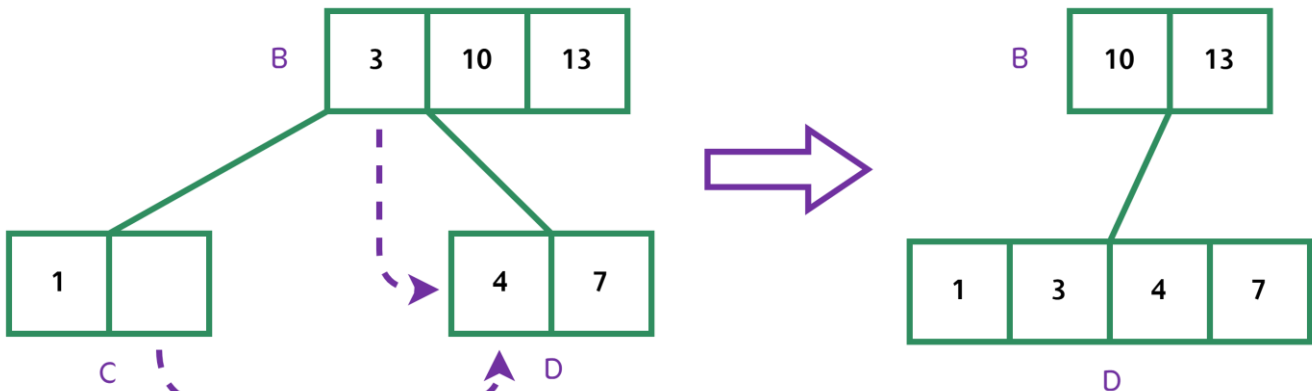


و يصبح الشكل بعد الحذف:

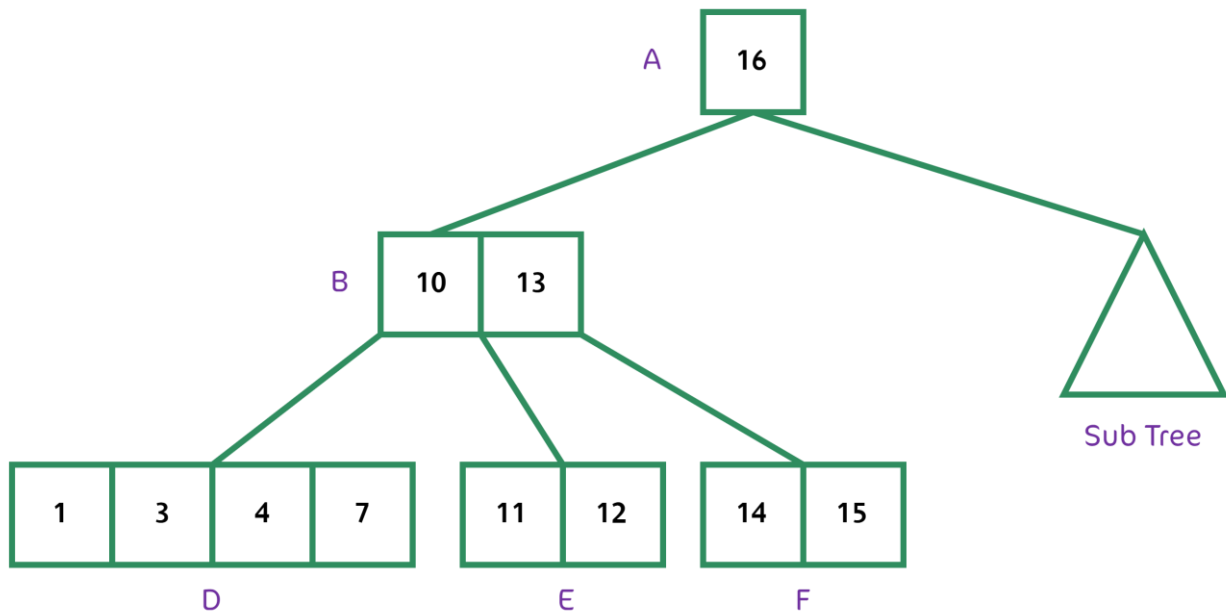


نريد حذف العنصر (2):

العنصر (2) موجود في الصفحة C و لكن هو ورقة تحوي t عنصر فقط ← بالتالي أخ الصفحة C هو D و لكن لا يحوي أكثر من t عنصر، بالتالي سنحذف العنصر (2) و ندمج العنصر (1) المتبقي من الصفحة C مع الأخ D بالإضافة لعنصر ينزل من الأب و يندمج معهم.

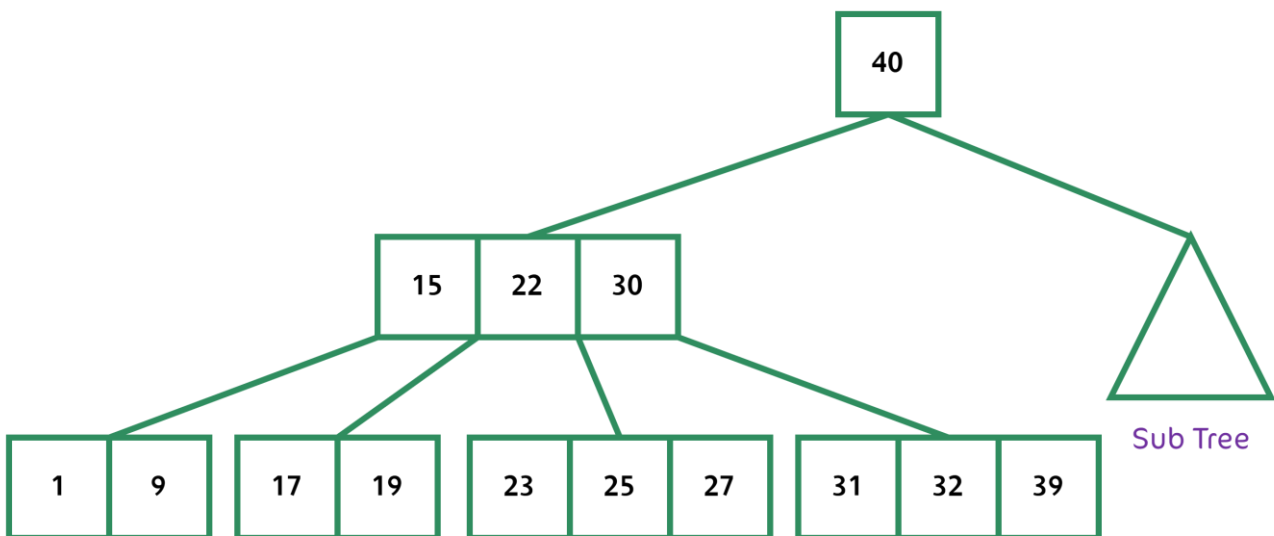


و يصبح الشكل بعد الحذف:

ثانياً: الحذف من صفحة داخلية:

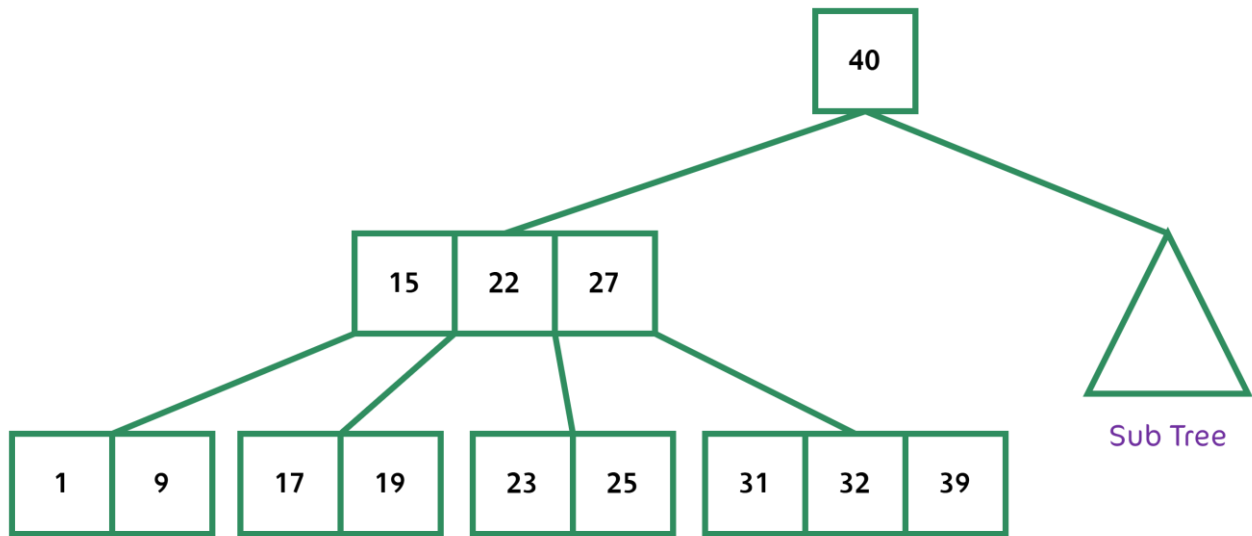
وهنا سنميز أربع حالات مرتبة حيث إذا لم تتحقق الأولى نلجأ للثانية وإذا لم تتحقق الثانية نلجأ للثالثة وهكذا ...

1. الحالة الأولى: العنصر الذي نريد حذفه يملك عنصر تالي أو سابق أحدهما قابل للحذف، نقوم بفحص إمكانية حذف العنصر السابق ثم العنصر التالي ثم نبذل الممكن منهم مع العنصر المراد حذفه ويتم الحذف.

مثال: لدينا الـ B-Tree التالية حيث $t = 2$:

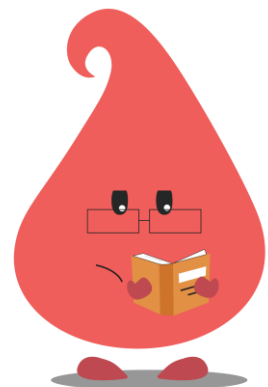
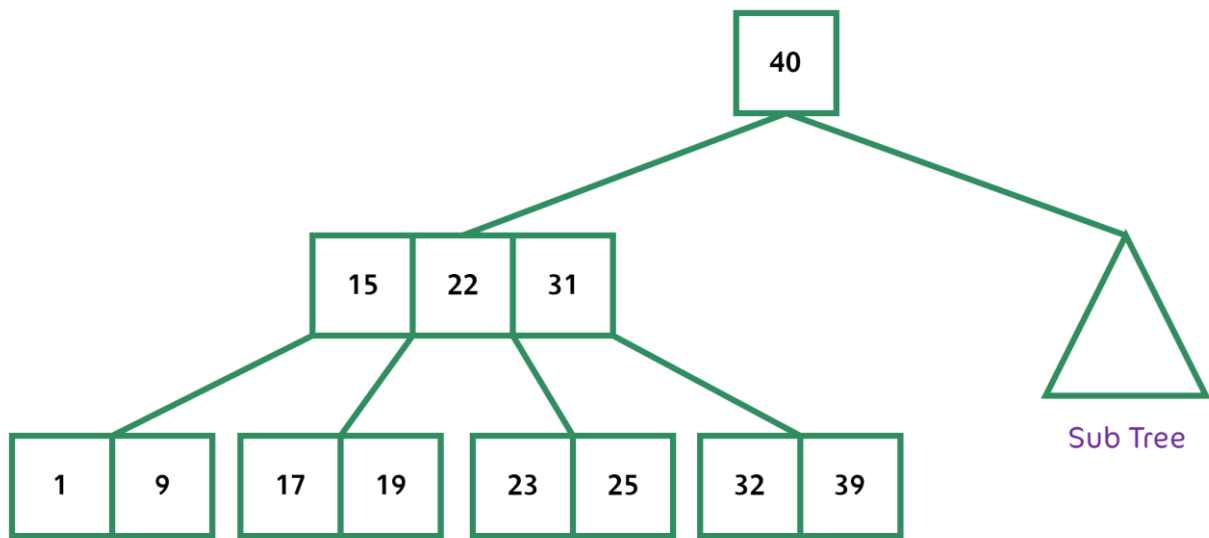
نريد حذف العنصر (30):

في الصفحة الداخلية نجد أنّ له عنصر سابق هو (27) نبدّل الـ (30) مع الـ (27) ونحذف الـ (30)، بالتالي:



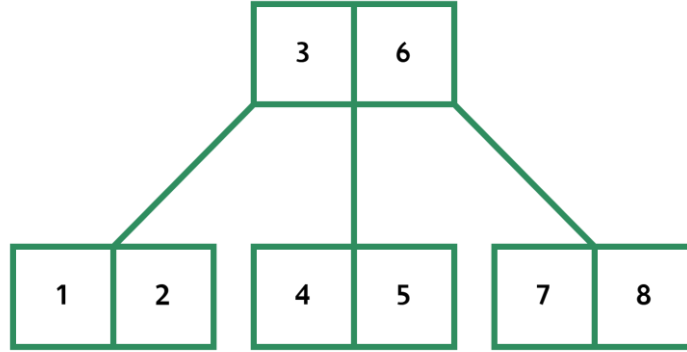
نريد حذف العنصر (27):

العنصر (27) في الصفحة ليس له عقدة سابقة قابلة للحذف لكن له عنصر تالي قابل للحذف وهو (31)؛ نبدّله مع (27) ونحذف العنصر (27).



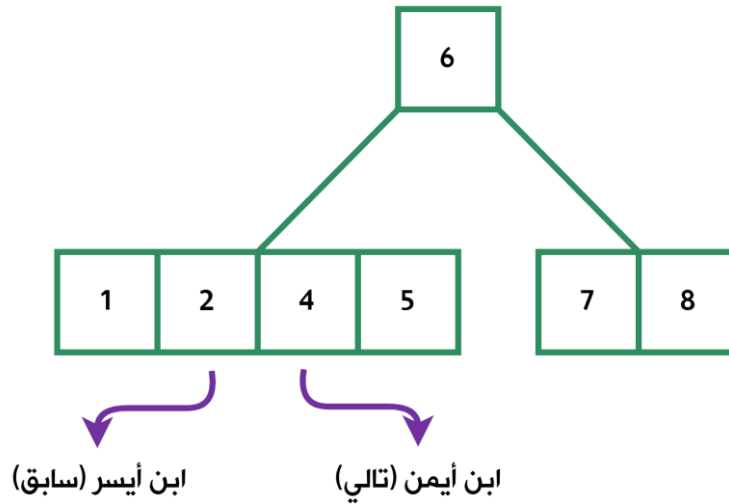
2. الحالة الثانية: الصفحتان التالية و السابقة للعنصر المراد حذفه تملكان t عنصر؛ أي غير قابلتين للحذف، لذا سنقوم بحذف العنصر ودمج ابنيه معاً وذلك في حالة كانت صفحة العنصر تحوي أكثر من t عنصر أو كانت تحوي عنصر ولكنها في صفحة الـ Root.

مثال: لدينا الـ B-Tree التالية حيث $t = 2$:

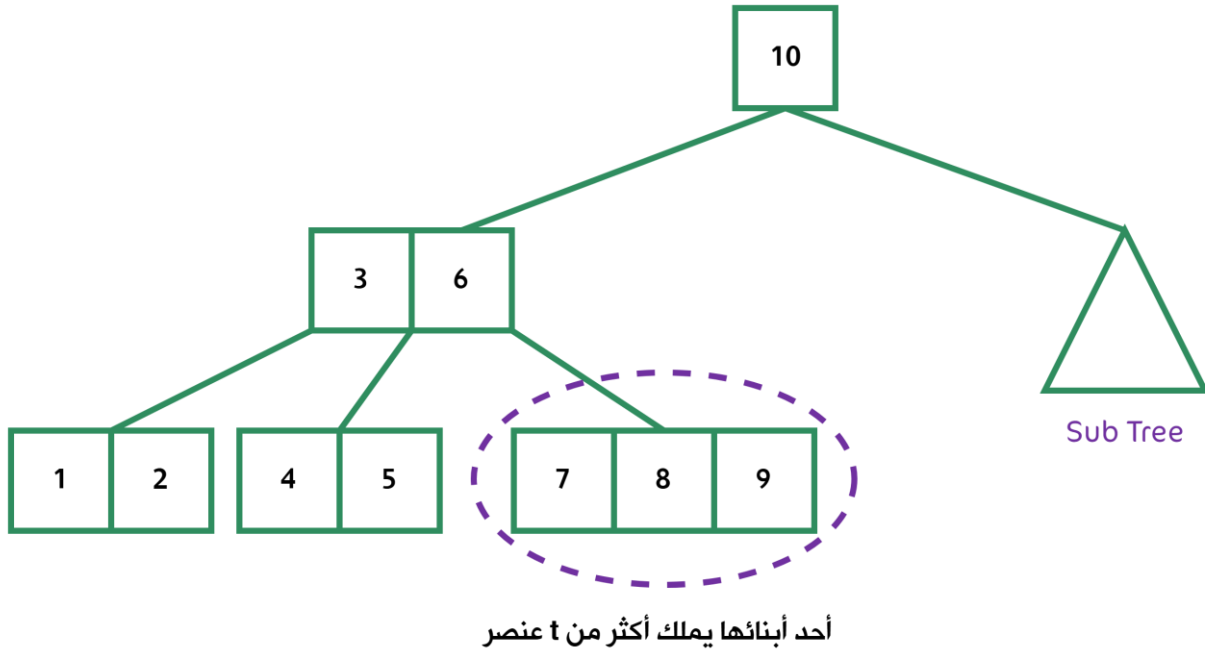


نريد حذف العنصر (3):

والعنصر (3) صفحة داخلية وابنها التالي والسابق لا يمكن تبديلهما "غير قابلين للحذف" و بالتالي سندمج بناءً على الحالة الثانية.

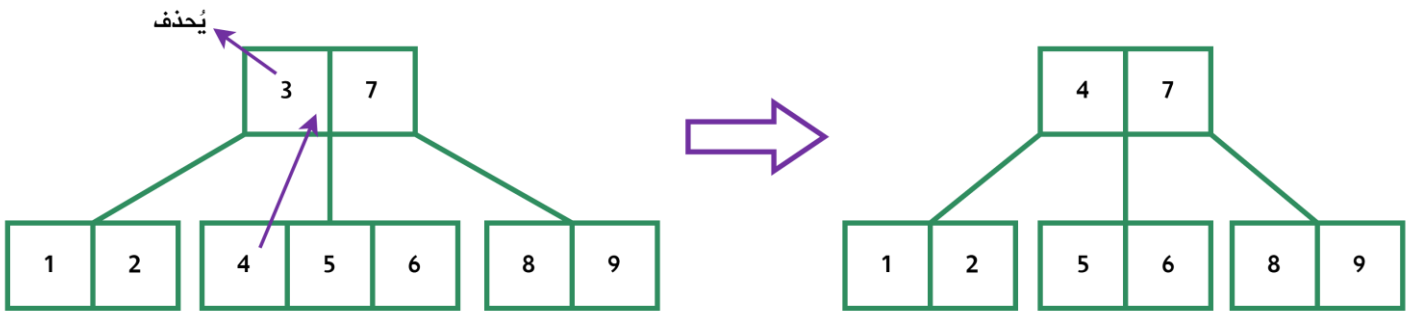


3. الحالة الثالثة: صفحة العنصر الذي نريد حذفه أحد أبنائها يملك أكثر من t فنقوم باستعارة عنصر من هذه الصفحة مع المحافظة على ترتيب العناصر (الابن الأيسر هو الأصغر والأيمن هو الأكبر) وذلك من خلال عمليات تبديل وإزاحة.

مثال: لدينا B-Tree التالية حيث $t = 2$:

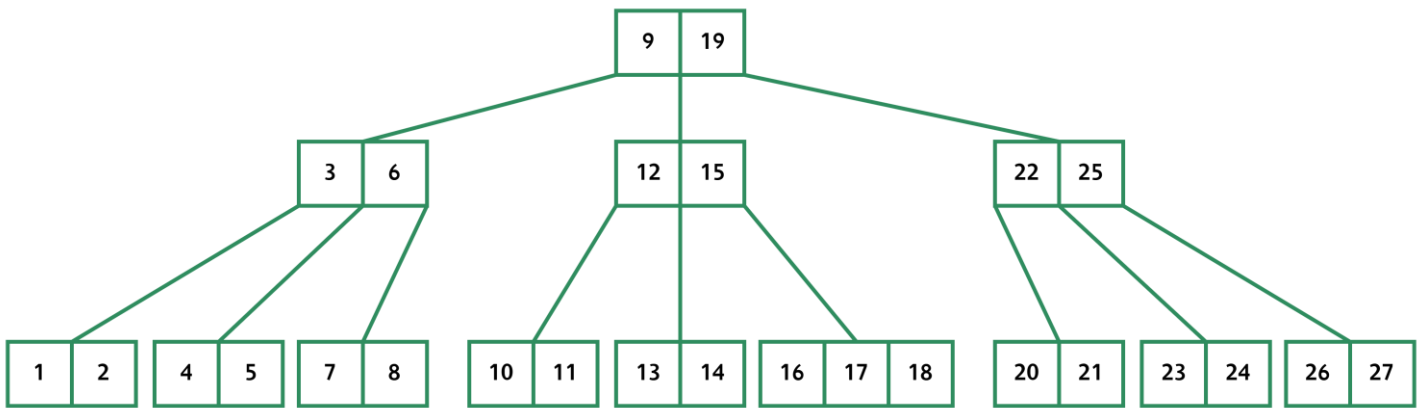
نريد حذف العنصر (3):

وهو عنصر داخليّ أحد أبناء صفحته يملك أكثر من t عنصر سنقوم بتغييرات واستعارة من هذه الصفحة ثم نحذف العنصر، لدينا (7) هي العنصر التالي لأحد أبناء صفحة العنصر المراد حذفه أي للعنصر (6) وبالتالي سنرفع الـ (7) للأب ومن ثمّ نضيف الـ (6) للصفحة المجاورة وهكذا أصبح بإمكاننا العودة للحالة الأولى واستبدال العنصر التالي مع العنصر المراد حذفه، فنحذفه ويصبح الشكل:



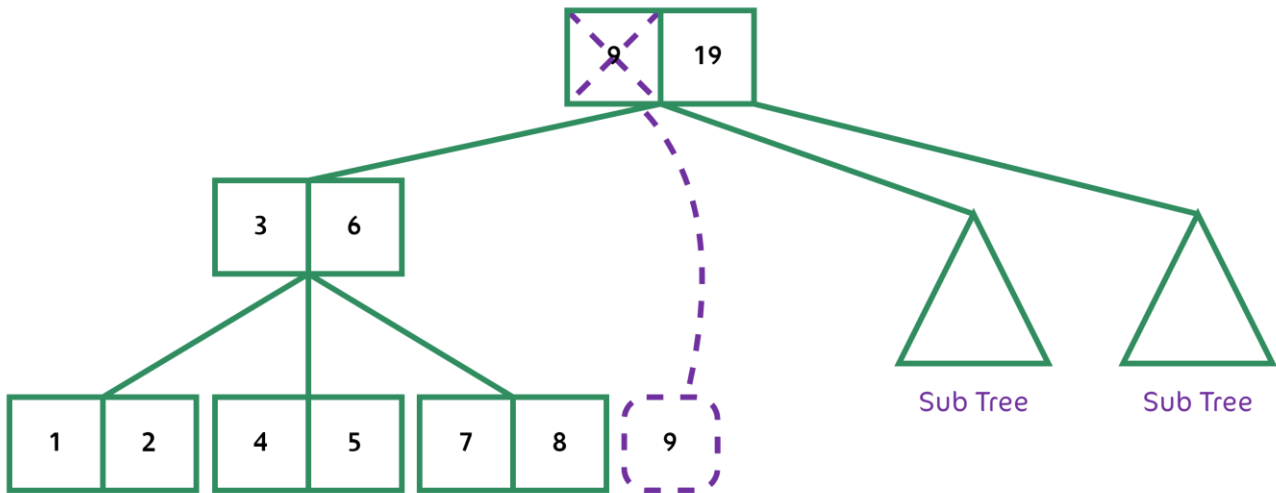
4. الحالة الرابعة: جميع أبناء وأحفاد صفحة العنصر تحوي t عنصر فقط، فنقوم باستعارة عنصر من الأب لنضعه في المكان الذي نحتاجه ضمن الـ SubTree الخاصة بالعنصر المراد حذفه وهكذا يمكننا إعادة الحالة لأحد الحالات الثلاث السابقة ونعامل العنصر المأخوذ من الأب كأننا حذفنا عنصر فنعالجه أيضاً بإحدى الحالات الأربعة.

ملاحظة هامة: عند وضع عنصر الأب في المكان المناسب فإنه سيكون ضمن ورقة إما في أقصى اليمين أو أقصى اليسار لأن الأب إما أن يكون أكبر من كل العناصر أو أصغر من كل العناصر.

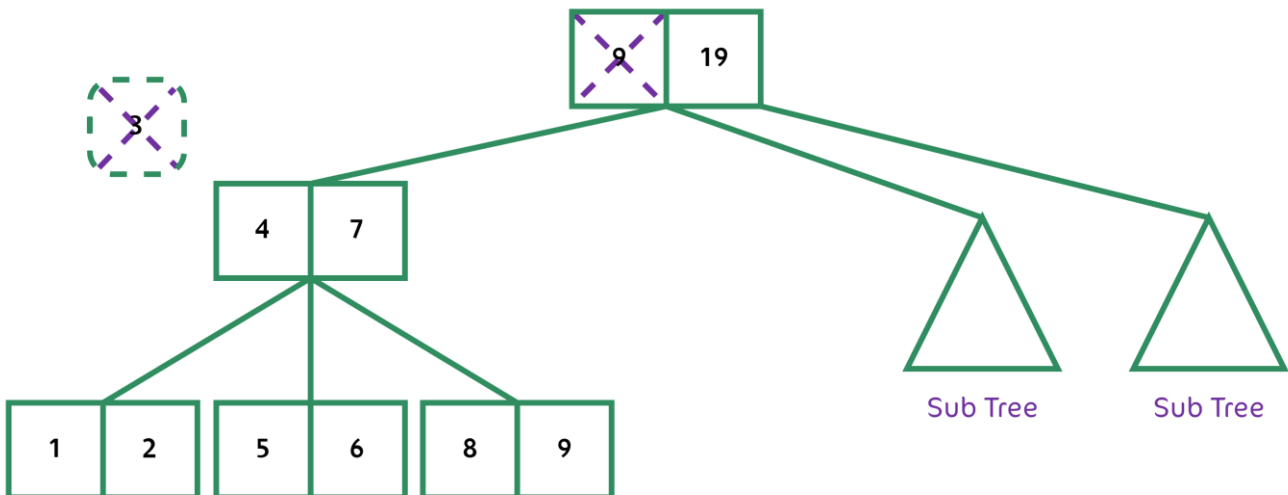
مثال: لدينا الـ B-Tree التالية حيث $t = 2$:

نريد حذف العنصر (3):

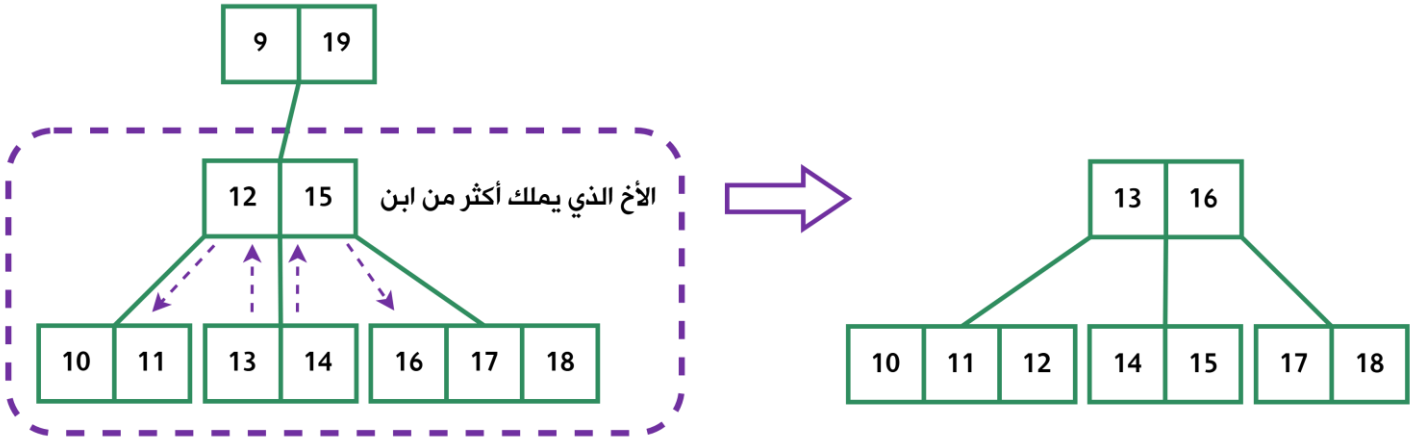
عنصر داخلي ولا نستطيع استبداله بالعنصر السابق أو التالي وكل أبنائه تحوي t عنصر وكل أبناء صفحته تحوي t عنصر
 ← حالة رابعة تتطلب عنصر من الأب ولنأخذ (9) ونضعها في مكانها المناسب ضمن الصفحة التي تحوي العنصرين (7) و (8) ونضعها بأقصى اليمين.



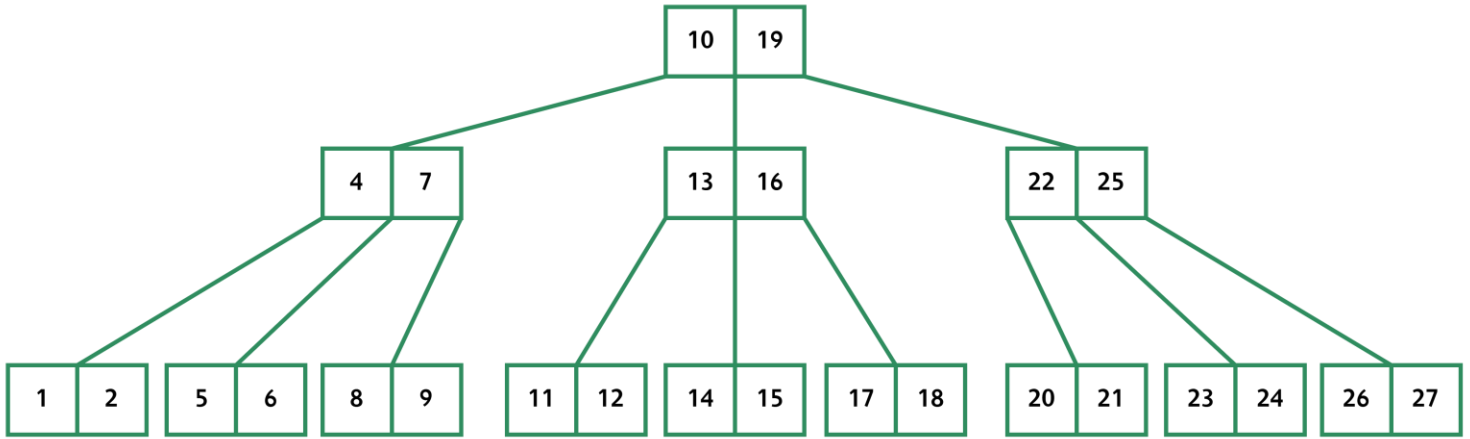
الآن عادت الشجرة لإحدى الحالات الثلاث الأولى: عنصر داخلي و أبنائها يملكان t عنصر و أحد أبنائها صفحتها يملك أكثر من t
 ← حالة ثالثة:



والآن تبقى لدينا معالجة حذف العنصر (9) من الجدّ: نبحث عن العنصر التالي أو السّابق و هما (8) و (10) على التوالي. مبدئياً العنصر السّابق يحوي t عنصر ولكن أحد إخوته يحوي أكثر من t عنصر.



و الآن أصبح العنصر التالي لـ (9) هو (10) و هو قابل للحذف البسيط كما تعلّمنا في الحالة الأولى.



يحسب تعقيد الحذف كما في الإضافة فيكون: $O(h \cdot t)$ أي $O(t \cdot \log_t(n))$ و بالتالي التعقيد النهائي هو: $O(\log_t(n))$.



المحاضرة مو سهلة

بس انت مهندس